



Scripts de Implantação

A imagem de contêiner, os serviços, as variáveis de ambiente e os comandos que sobem a Plataforma de Dados MinC, com o que cada peça faz e o que precisa ser definido antes de rodar.

Universidade de Brasília

Lab Livre · Faculdade do Gama, UnB

Ministério da Cultura

Gov Hub BR

Projeto de Pesquisa

Plataforma de Dados do Ministério da Cultura · Gov Hub BR

Meta 03 · Produto 3

Scripts e procedimentos de implantação.

Documento

Scripts de Implantação



Índice

01	Objetivo e escopo	4
	Os artefatos	4
02	A construção do Airflow	5
	Certificados da ICP-Brasil	5
03	Os serviços do Compose	6
	Ordem de inicialização	6
04	Variáveis de ambiente	8
	Valores padrão que servem só a desenvolvimento	9
05	Comandos e automação	10
	Comandos do Makefile	10
	Construção e publicação automatizadas	10
	Encerramento de escopo	11





Este documento descreve os artefatos de implantação versionados no repositório: o que cada um constrói, quais variáveis consome e em que ordem os serviços sobem.

Os artefatos

Caminho	O que é
<code>infra/docker/airflow/Dockerfile</code>	Imagem do Airflow, em três estágios.
<code>infra/docker/airflow-mcp/Dockerfile</code>	Imagem do servidor MCP de operação do Airflow.
<code>infra/docker/superset/Dockerfile</code>	Imagem do Superset.
<code>infra/docker/postgres/init.sh</code>	Criação das bases na primeira subida.
<code>infra/docker-compose.yml</code>	Sete serviços e dois volumes.
<code>infra/airflow/airflow.cfg</code>	Configuração do Airflow, montada no contêiner.
<code>infra/trino/etc/</code>	Configuração do Trino: nó, JVM e um catálogo por schema de origem.
<code>infra/env/.env.example</code>	Modelo de variáveis, com valores de desenvolvimento.
<code>Makefile</code>	Comandos de ciclo de vida.
<code>requirements.txt</code>	Dependências instaladas na imagem.
<code>.github/workflows/main.yml</code>	Construção e publicação automatizadas.

ESTE DOCUMENTO DESCREVE O AMBIENTE LOCAL

O `docker-compose.yml` define o ambiente de desenvolvimento. Não existe, no momento, ambiente de produção: o ambiente hoje disponível é de homologação, e a topologia definitiva permanece em estudo, orientada pela diretriz de que a totalidade dos dados resida na infraestrutura do próprio Ministério da Cultura. A arquitetura física é, por isso, objeto de outro produto, e não deste.





O `Dockerfile` tem três estágios. O primeiro prepara o sistema operacional; os outros dois só diferem na variável `ENVIRONMENT`, o que permite construir a mesma imagem para desenvolvimento e para produção sem duplicar instruções.

Estágio	O que faz
<code>airflow-base</code>	Parte da imagem oficial do Airflow na versão declarada. Instala bibliotecas de sistema, gera as localidades <code>en_US</code> e <code>pt_BR</code> , e copia o arquivo de senhas do gerenciador de autenticação.
<code>airflow-prod</code>	Instala as dependências, confere se a versão do Airflow instalada é a esperada, roda <code>pip check</code> e define <code>ENVIRONMENT=prod</code> .
<code>airflow-dev</code>	Idêntico ao anterior, com <code>ENVIRONMENT=dev</code> . É o alvo padrão do Compose.

Certificados da ICP-Brasil

A imagem baixa e instala a cadeia de certificados da ICP-Brasil, e rebaixa o nível mínimo de TLS e a política de cifras do OpenSSL. Isso não é descuido: vários serviços de governo apresentam certificados que a configuração padrão do Debian recusa, e sem esse ajuste as chamadas de API falham no aperto de mão TLS. A mesma razão está por trás das opções de conexão do catálogo do SALIC no Trino, que desabilitam a cifragem exigida por padrão pelo driver moderno diante de um SQL Server antigo.

VERIFICAÇÃO EMBUTIDA NA CONSTRUÇÃO

Os dois estágios finais conferem, dentro do próprio `RUN`, se a versão instalada do Airflow é igual à versão pedida, e executam `pip check`. Uma incompatibilidade de dependência quebra a construção da imagem, e não a execução da DAG.





Serviço	Imagem ou origem	Porta	Papel
<code>postgres</code>	<code>postgres:15-alpine</code>	5432	Base de metadados do Airflow e base analítica da plataforma.
<code>airflow</code>	Construída localmente	8080	Agendador, interface e execução, em modo <code>standalone</code> .
<code>trino</code>	<code>trinodb/trino</code>	8090	Motor de consulta distribuído. Lê do SQL Server do SALIC e grava no Postgres, sem que o dado passe por Python.
<code>airflow-mcp</code>	Construída localmente	8000	Servidor MCP para operar o Airflow. Exposto só em <code>127.0.0.1</code> .
<code>superset</code>	Construída localmente	8088	Visualização e exploração dos dados.
<code>minio</code>	<code>minio/minio</code>	9000, 9001	Armazenamento de objetos: pouso dos anexos e dos lançamentos.
<code>minio-init</code>	<code>minio/mc</code>	—	Cria os buckets na primeira subida e encerra.

Dois volumes nomeados persistem o estado: `postgres-db` e `minio-data`.

O TRINO SOBE POR PERFIL PRÓPRIO

O serviço do Trino é declarado sob o perfil `trino` do Compose e, por isso, não sobe junto com `make up`: exige `make up-trino`. A separação é deliberada — o motor de consulta dimensiona a memória do próprio contêiner e depende de acesso à rede da origem, condições que nem toda sessão de desenvolvimento precisa satisfazer.

Ordem de inicialização

O `postgres` sobe primeiro e executa `init.sh`, que cria as bases `airflow` e `data_warehouse`. Esse script roda uma única vez, na criação do volume: se o volume já existe, ele não é reexecutado. Os demais serviços declaram dependência do banco e só sobem depois que ele responde.



```
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" \  
    --dbname "$POSTGRES_DB" <<-EOSQL  
CREATE DATABASE airflow;  
CREATE DATABASE data_warehouse;  
EOSQL
```

[infra/docker/postgres/init.sh](#)

UMA RESSALVA QUANTO AO NOME DA BASE

O nome `data_warehouse` é rótulo herdado do ambiente local e não descreve a arquitetura adotada: a plataforma é concebida como **data lakehouse**, com armazenamento em object storage sob formato de tabela aberta e leitura por motor de consulta distribuído. O banco relacional cumpre hoje o papel de destino da transformação enquanto a arquitetura-alvo não é implantada.





O Compose lê `infra/.env`. O modelo versionado é `infra/env/.env.example`, com valores de desenvolvimento. Nenhum arquivo com valor real de credencial entra no repositório.

Variável	Papel
<code>AIRFLOW_VERSION</code>	Versão da imagem base. Conferida durante a construção.
<code>AIRFLOW_BUILD_TARGET</code>	Estágio a construir. Padrão <code>airflow-dev</code> .
<code>AIRFLOW_UID</code>	Usuário do contêiner. Alinha a propriedade dos arquivos montados.
<code>POSTGRES_USER</code> · <code>POSTGRES_PASSWORD</code>	Credenciais da base.
<code>AIRFLOW__CORE__FERNET_KEY</code>	Chave que cifra senhas e campos sensíveis das Connections.
<code>AIRFLOW__API__SECRET_KEY</code>	Segredo de autenticação entre os subprocessos.
<code>AIRFLOW__API__AUTH__JWT_SECRET</code>	Segredo de assinatura dos tokens internos.
<code>AIRFLOW_PRUNE_STALE_DAGS</code>	Remove DAGs que sumiram do repositório.
<code>MINIO_ACCESS_KEY</code> · <code>MINIO_SECRET_KEY</code>	Credenciais do armazenamento de objetos.
<code>TRINO_VERSION</code> · <code>TRINO_PORT</code> · <code>TRINO_MEM_LIMIT</code>	Versão, porta publicada e teto de memória do motor de consulta.
<code>SALIC_MSSQL_*</code> · <code>SALIC_DNS_SEARCH</code>	Endereço, credenciais e domínio de busca da origem do SALIC. Sem valor padrão útil: vazios deixam o contêiner subir, e é a consulta que falha, com erro de conexão claro.

DUAS VARIÁVEIS QUE EXIGEM ATENÇÃO ANTES DE QUALQUER IMPLANTAÇÃO REAL

A **chave Fernet** nunca deve ser trocada sem rotação. Substituí-la sem executar `airflow rotate-fernet-key` torna toda Connection e Variable sensível existente indecifrável, exigindo apagar e recriar tudo. O aviso está registrado no próprio arquivo de exemplo porque o problema já ocorreu no projeto. Os **segredos de API e JWT** não podem ficar vazios: se ficarem, cada subprocesso do Airflow gera um valor próprio e as tarefas falham na verificação de assinatura.



Valores padrão que servem só a desenvolvimento

Para que a subida local funcione sem configuração, o Compose define valores padrão para alguns segredos. Eles cumprem o papel de reduzir atrito no ambiente de desenvolvimento, e precisam ser substituídos antes de qualquer implantação com dado real.

Item	Situação	Ação antes de produção
Chave Fernet	Valor padrão embutido no Compose.	Gerar chave própria e definir no <code>.env</code> .
Segredos de API e JWT	Valores padrão de desenvolvimento.	Gerar com <code>openssl rand -hex 32</code> .
Usuário do Airflow	<code>airflow:admin</code> pelo gerenciador simples.	Substituir por autenticação institucional.
Chave do Superset	Valor fixo no Compose.	Mover para o <code>.env</code> e gerar valor próprio.
Credenciais do Postgres	<code>postgres/postgres</code> no exemplo.	Definir credenciais reais no <code>.env</code> .

FUSO HORÁRIO DEFINIDO EXPLICITAMENTE

`AIRFLOW__CORE__DEFAULT_TIMEZONE` e o fuso da interface estão fixados em `America/Sao_Paulo`. É o mesmo fuso do carimbo `dt_transform` do catálogo de metadados, o que evita divergência entre o horário de execução registrado no Airflow e o horário gravado na tabela.





Comandos do Makefile

Comando	O que faz
<code>make setup</code>	Instala o Poetry, resolve as dependências, exporta o arquivo de requisitos e instala os ganchos de git.
<code>make up</code>	Sobe banco, Airflow e servidor MCP.
<code>make up-trino</code>	Sobe também o motor de consulta, que está sob perfil próprio.
<code>make down</code>	Encerra os serviços.
<code>make logs-airflow</code> · <code>make logs-trino</code>	Mostram as últimas linhas de cada serviço.
<code>make trino-cli</code>	Abre o cliente de linha de comando do motor de consulta.
<code>make compose-config</code>	Valida o Compose, resolvendo variáveis, sem subir nada.
<code>make format</code>	Aplica black, ruff e sqlfmt.
<code>make lint</code>	Confere black, ruff, mypy, sqlfmt e sqlfluff.
<code>make test</code>	Executa a suíte de testes.
<code>make docs-collect</code> · <code>docs-build</code> · <code>docs-serve</code>	Coletam o acervo, constroem e servem o site de documentação.

VALIDAR ANTES DE SUBIR

`make compose-config` resolve todas as variáveis e imprime o Compose final sem iniciar contêiner nenhum. É a forma mais barata de descobrir que falta uma variável no `.env`.

Construção e publicação automatizadas

O fluxo em `.github/workflows/main.yaml` tem quatro etapas encadeadas. A publicação da imagem só acontece em integração na ramificação principal.



Etapa	Quando	O que faz
<code>lint</code>	Todo PR e toda integração	Executa <code>make lint-ci</code> , que confere apenas o SQL.
<code>test</code>	Todo PR e toda integração	Executa a suíte com relatório de cobertura, publicado como artefato.
<code>docker_build</code>	Depois de <code>lint</code> e <code>test</code>	Constrói o estágio <code>airflow-prod</code> com cache, sem publicar.
<code>docker_push</code>	Só na ramificação principal	Publica no registro do GitHub com as etiquetas do commit e <code>latest</code> .

O QUE A AUTOMAÇÃO HOJE NÃO IMPEDE

A etapa de `lint` termina com `|| true`, por decisão registrada da equipe: ela informa, mas não reprova o PR. Além disso, `make lint-ci` cobre só SQL, de modo que `black`, `ruff` e `mypy` não são executados na automação, embora estejam configurados e sejam exigidos pelo `make lint` local. A suíte de testes é executada com medição de cobertura, mas sem limite mínimo. Na prática, a única etapa que reprova uma integração hoje é a construção da imagem.

Encerramento de escopo

Procedimento de retorno a versão anterior, testes de verificação pós-implantação e política de cópia de segurança não constam do repositório e, por isso, não estão documentados aqui. Eles pertencem ao roteiro de implantação em produção, que depende de decisões de infraestrutura ainda não tomadas.

